

LPOO — V4: Aprendendo a Resolver e Calcular do Zero

Esta versão transforma a revisão em um roteiro de execução. A ideia não é decorar palavras: é olhar o código, descobrir o que acontece e acompanhar o resultado.

0. O que significa “calcular” em LPOO?

Em LPOO, muitas questões não pedem uma conta matemática. “Calcular” significa descobrir o que o programa fará.

Você pode precisar descobrir: - qual construtor será chamado; - qual valor uma variável terá; - qual método será executado; - se há erro de compilação; - se ocorre exceção; - qual será a saída; - se é overload ou override; - como uma coleção trata um objeto.

Use sempre:

IDENTIFIQUE → SIGA O CÓDIGO → DECIDA → EXECUTE MENTALMENTE → CONFIRA

1. Transforme o código em uma história

Considere:

```
Livro l = new LivroDigital("Duna", "Herbert", 1965, 5.2);
```

Leia assim: 1. a variável se chama l; 2. a referência é do tipo Livro; 3. o objeto real é LivroDigital; 4. o construtor de LivroDigital começa; 5. super(...) chama a parte da classe mãe; 6. depois o construtor da filha termina.

A tradução do código para uma sequência de acontecimentos é a principal técnica desta revisão.

2. Encapsulamento: como descobrir o valor do atributo

Exemplo:

```
class Produto {  
    private double preco;  
  
    public void setPreco(double preco) {  
        if (preco < 0) {  
            throw new IllegalArgumentException();  
        }  
        this.preco = preco;  
    }  
  
    public double getPreco() {  
        return preco;  
    }  
}
```

```
Produto p = new Produto();
```

```
p.setPreco(50);  
System.out.println(p.getPreco());
```

Passo a passo

50 < 0 é falso.

Então executa:

```
this.preco = 50;
```

Depois getPreco() devolve 50.

Resultado: 50.

Agora pense em setPreco(-10):

-10 < 0 é verdadeiro.

Então ocorre IllegalArgumentException e a atribuição não acontece.

Regra

SETTER COM VALIDAÇÃO = TESTE A CONDIÇÃO ANTES DA ATRIBUIÇÃO.

3. Construtores: descubra a ordem

Exemplo:

```
class Animal {  
    Animal() {  
        System.out.println("Animal");  
    }  
}  
  
class Cachorro extends Animal {  
    Cachorro() {  
        super();  
        System.out.println("Cachorro");  
    }  
}
```

Executando new Cachorro():

1. entra no construtor de Cachorro;
2. executa super();
3. entra em Animal;
4. imprime Animal;
5. volta para Cachorro;
6. imprime Cachorro.

Resultado:

Animal

Cachorro

Regra

PAI PRIMEIRO → FILHO DEPOIS.

4. Herança: this e super

Exemplo:

```
class Veiculo {
    protected int velocidade = 50;
}

class Carro extends Veiculo {
    int velocidade = 100;

    void mostrar() {
        System.out.println(velocidade);
        System.out.println(super.velocidade);
    }
}
```

Dentro de Carro:

- velocidade procura o campo da classe atual → 100;
- super.velocidade força o campo da mãe → 50.

Resultado:

```
100
50
```

Regra

this = contexto atual.

super = parte herdada da superclasse.

5. Overload: como descobrir qual método será chamado

Exemplo:

```
class Calculadora {
    int somar(int a, int b) {
        return a + b;
    }

    double somar(double a, double b) {
        return a + b;
    }
}
```

Para somar(2, 3):

- parâmetros = int, int;
- escolhe somar(int, int);
- $2 + 3 = 5$.

Para somar(2.5, 3.5):

- parâmetros = double, double;
- escolhe somar(double, double);
- $2.5 + 3.5 = 6.0$.

Regra

OVERLOAD = COMPARE OS PARÂMETROS.

A decisão é feita em compilação.

6. Override e polimorfismo

Exemplo:

```
class Animal {
    void emitirSom() {
        System.out.println("Animal");
    }
}
```

```
class Gato extends Animal {
    @Override
    void emitirSom() {
        System.out.println("Miau");
    }
}
```

```
Animal a = new Gato();
a.emitirSom();
```

O erro comum é pensar:

“a variável é Animal, então imprime Animal”.

Faça assim: 1. tipo da variável = Animal; 2. objeto real = Gato; 3. o método foi sobrescrito? Sim; 4. então, em execução, entra a versão de Gato.

Resultado:

Miau

Regra

PARA MÉTODO SOBRESCRITO, OLHE O OBJETO REAL.

7. abstract: como resolver questões de instanciação

Exemplo:

```
abstract class Forma {  
    abstract double calcularArea();  
}
```

Isto é inválido:

```
Forma f = new Forma();
```

Porque classe abstrata não pode ser instanciada diretamente.

Agora:

```
class Quadrado extends Forma {  
    private double lado;  
  
    Quadrado(double lado) {  
        this.lado = lado;  
    }  
  
    @Override  
    double calcularArea() {  
        return lado * lado;  
    }  
}
```

```
Forma f = new Quadrado(4);
```

A referência é Forma, mas o objeto é Quadrado.

Cálculo:

$4 \times 4 = 16$.

Resultado: 16.

Regra

ABSTRACT = PODE SER TIPO DE REFERÊNCIA, NÃO PODE SER OBJETO DIRETO.

8. static: como acompanhar um valor compartilhado

Exemplo:

```
class Compra {  
    static int contador = 0;  
  
    Compra() {  
        contador++;  
    }  
}
```

Executando três construtores:

Estado inicial: contador = 0

Primeiro: $0 + 1 = 1$

Segundo: $1 + 1 = 2$

Terceiro: $2 + 1 = 3$

Resultado:

contador = 3.

Regra

STATIC É COMPARTILHADO PELA CLASSE.

9. final: referência não é o mesmo que conteúdo

Exemplo:

```
final int x = 10;
```

$x = 20$ é inválido.

Agora:

```
final List<String> nomes = new ArrayList<>();
```

`nomes.add("Lucas")` pode funcionar.

Mas:

```
nomes = new ArrayList<>();
```

é inválido.

Regra

FINAL IMPEDE REATRIBUIÇÃO DA REFERÊNCIA; NÃO SIGNIFICA IMUTABILIDADE DO OBJETO.

10. Exceções: acompanhe o fluxo

Exemplo:

```
try {  
    int x = 10 / 0;  
    System.out.println(x);  
} catch (ArithmeticException e) {  
    System.out.println("Erro");  
} finally {  
    System.out.println("Fim");  
}
```

10 / 0 gera `ArithmeticException`.

Logo: - a linha que imprime x não executa; - o catch executa; - depois finally executa.

Saída:

Erro

Fim

Regra

TRY TENTA → CATCH TRATA → FINALLY FINALIZA.

11. throw x catch

throw significa lançar uma exceção.

catch significa capturar uma exceção.

Não confunda:

THROW = DISPARA.

CATCH = RECEBE/TRATA.

12. equals: como comparar objetos

Considere:

```
Livro a = new Livro("Duna");  
Livro b = new Livro("Duna");
```

Sem equals sobrescrito, dois objetos diferentes podem resultar em:

```
a.equals(b) == false
```

mesmo possuindo os mesmos dados.

Compare as ideias: - `==` trabalha com a referência; - equals representa igualdade lógica quando a classe redefine o método.

13. hashCode: a regra que precisa ficar automática

Regra obrigatória:

a.equals(b) verdadeiro implica a.hashCode() == b.hashCode()

Mas hash igual não significa necessariamente equals verdadeiro.

Ao usar HashSet ou HashMap, pense em equals e hashCode juntos.

14. Coleções: descubra o comportamento

List

Aceita repetição e preserva ordem.

Se adicionar A e depois A, há dois elementos.

Set

Representa um conjunto sem repetição.

Map

Funciona como chave → valor.

Exemplo:

```
Map<String, Integer> idade = new HashMap<>();  
idade.put("Ana", 20);
```

Então:

```
idade.get("Ana")
```

devolve 20.

Regra

LIST = ORDEM. SET = NÃO REPETIR. MAP = CHAVE → VALOR.

15. Modificadores de acesso

Quando aparecer uma questão de acesso, faça duas perguntas:

1. Quem está tentando acessar?
2. De onde está tentando acessar?

Regra básica: - private → própria classe; - acesso padrão → mesmo pacote; - protected → própria classe, pacote e subclasses nos casos permitidos; - public → acesso amplo.

Comece pensando em private e só abra quando a situação exigir.

16. instanceof

Exemplo:

```
Animal a = new Gato();
```

```
a instanceof Animal  
a instanceof Gato
```

Os dois testes resultam em true porque o objeto real é Gato e Gato é um Animal.

Regra

INSTANCEOF TESTA O OBJETO REAL.

17. Como calcular a saída do console

Monte uma tabela.

Passo	Variável	Objeto real	Ação	Saída
1	a	Gato	criação	—
2	a	Gato	emitirSom()	Miau

Não tente guardar toda a execução na cabeça.

18. Como resolver alternativa de prova

Etapa 1 — procure erro de compilação

Exemplos: - instanciar classe abstract; - tentar alterar final; - assinatura errada de override; - método inexistente; - catch impossível.

Etapa 2 — siga o fluxo

Quem chama quem?

Etapa 3 — faça os cálculos

Retornos, comparações, incrementos e valores.

Etapa 4 — faça uma segunda passagem

Confira a parte mais importante.

19. DEU MERDA — memória

Overload x Override

ERRO: “mesmo nome = override”.

CONSEQUÊNCIA: classificação errada.

REGRA: parâmetros diferentes → overload; mesma assinatura sobrescrita → override.

RESULTADO: overload é decidido na compilação; override participa do despacho em execução.

IDEIA CENTRAL DE LPOO

Quando der branco, pergunte:

QUAL É A CLASSE? QUAL É O OBJETO? QUAL CONSTRUTOR RODA? QUAL MÉTODO É ESCOLHIDO? QUAL VALOR SOBRA?

20. Checklist

- ☐ Sei acompanhar construtores.
- ☐ Sei diferenciar `this` e `super`.
- ☐ Sei overload e override.
- ☐ Sei acompanhar polimorfismo.
- ☐ Sei `abstract`.
- ☐ Sei `static` e `final`.
- ☐ Sei `try/catch/finally`.
- ☐ Sei `throw` e `catch`.
- ☐ Sei `==` e `equals()`.
- ☐ Sei a relação `equals/hashCode`.
- ☐ Sei `List`, `Set` e `Map`.
- ☐ Sei `instanceof`.
- ☐ Consigo calcular uma saída linha por linha.

Fórmula de sobrevivência: em LPOO, não decore a saída; simule o caminho que produz a saída.